Hands-On Machine Learning

Chapter 3 — Classification

Precision · Recall · ROC · Multiclass · Multilabel

ملخص كشكولك + تكملة النقط الناقصة + رسومات وأكواد للمراجعة

من كشكولك | إضافة لإكمال النقص | انتبه / تصحيح

MNIST (بتاعة الكتاب) مش بتتنزّل في بيئة التشغيل هنا، فمربعات **Output (real run)** الخضرا طلعت من تشغيل فعلي على **digits** الحقيقية بتاعة sklearn (1797 صورة 8×8): نفس الفكرة بس مشكلة أسهل، فالأرقام أحسن من MNIST. أرقام MNIST بتاعة الكتاب (الطبعة الثالثة) مكتوبة في مربعات برتقالي ومعلَّمة «من الكتاب».

1 مجموعة بيانات MNIST

من كشكولك

أول حاجة في الشابتر ده هنستخدم MNIST (مجموعة بيانات)، وهي عبارة عن مجموعة من **70,000** صورة صغيرة للأرقام مكتوبة بخط اليد (من خلال ناس كتير). كل صورة بتمثّل رقم (من 0 لـ 9)، وكل صورة `28 × 28` = **784 بكسل**، وكل بكسل قيمة رقمية من 0 لـ 255 (شدة لون البكسل: من 0 أبيض لحد 255 أسود).

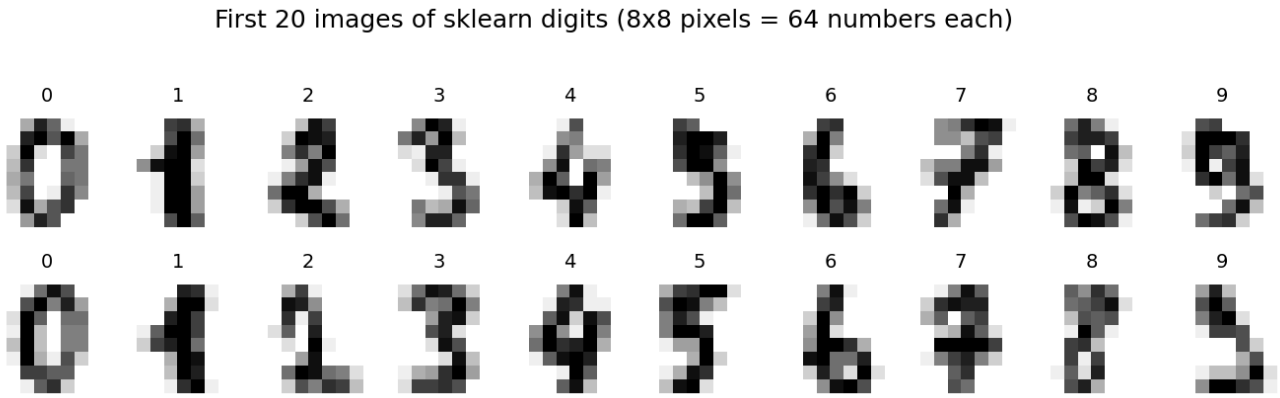
ملحوظة: في `sklearn.datasets` في 3 أنواع دوال: `fetch_*` (تحميل بيانات حقيقية كبيرة)، `load_*` (بيانات صغيرة جاهزة)، `make_*` (بيانات مصطنعة للاختبار).

```
from sklearn.datasets import fetch_openml
mnist = fetch_openml('mnist_784', as_frame=False)    # as_frame=False: return NumPy arrays, not a
DataFrame
X, y = mnist.data, mnist.target
X_train, X_test, y_train, y_test = X[:60000], X[60000:], y[:60000], y[60000:]
```

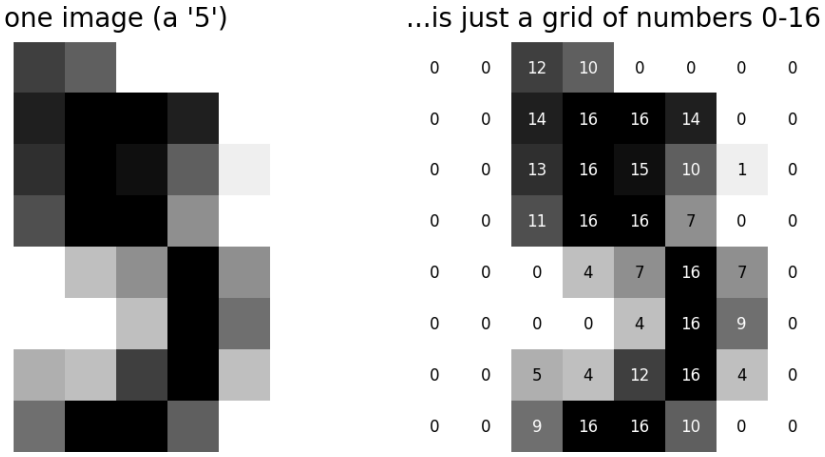
من كشكولك

`as_frame=False` عشان بيانات الصور بتتعامل أحسن كـ NumPy arrays (الـ DataFrame مش مناسب لصور). الـ MNIST **متقسّمة ومخلوطة مسبقًا**: أول 60,000 = تدريب، وآخر 10,000 = اختبار. الخلط مهم لأن بعض الخوارزميات حساسة لترتيب البيانات، وبتشتغل وحشة لو جاب كذا مثال متشابه ورا بعض.

`X_train` = صور التدريب، `X_test` = صور الاختبار، `y_train` = إجابات التدريب (labels)، `y_test` = إجابات الاختبار.



الأمثلة هنا من `digits` الحقيقية (8×8) بدل MNIST عشان التوضيح؛ MNIST صورها 28×28 والفكرة نفسها.



الصورة مجرد شبكة أرقام: كل رقم بيمثّل شدة بكسل (هنا 0-16؛ في MNIST من 0-255).

إضافة

- الـ `y` في MNIST بيرجع **نصوص** (`'5'` مش `5`)، عشان كده في كودك بتكتب `y_train == '5'`. ولو عايز أرقام: `y.astype(np.uint8)`.
- كل صف من `X` هو صورة واحدة مفرودة في vector طوله 784. ولو عايز تعرض صورة: `some_digit.reshape(28, 28)` وبعدها `plt.imshow(..., cmap="binary")`.
- الـ MNIST اتسمّى "hello world" للـ ML، لأن كل واحد بيتعلم classification بيجرب خوارزمية جديدة عليها.
- في التشغيل الحقيقي هنا: `digits` فيها **1797** صورة، قسّمناها 1437 تدريب و 360 اختبار (stratified).

2 تدريب Binary Classifier (5 أو مش 5)

من كشكولك

في الشابتر اللي فات الموديل كان بيتوقع رقم (سعر البيت، Regression). هنا في Classification الموديل بيتوقع **فئة** (class / category). يعني عايز الموديل يتوقع: الرقم ده 5 ولا مش 5؟ وهكذا. وBinary classification لأنه بيرد بـ `True` أو `False`.

خطوات:

1. **تبسيط المشكلة:** بدل ما الموديل يعرف كل الـ 10 أرقام (0 لـ 9) هنخليه يجاوب على سؤال واحد: هل الصورة رقم 5 ولا لأ؟ يعني نحاول نكتشف رقم واحد فقط (نعتبره 5 يبقى `True` وغير كده `False`).

2. **إنشاء الـ Target الجديد.**

```
y_train_5 = (y_train == '5')    # True for 5, False for any other digit
y_test_5  = (y_test  == '5')
```

من كشكولك

هنا بنحوّل الـ labels الأصلية (0, 1, 2, 3, ..., 9) لـ `True` (لو الصورة 5) و `False` (لو أي رقم تاني). فالموديل مش محتاج يعرف الرقم بالتحديد، محتاج يعرف بس هل هو 5 ولا مش 5 (لأننا دلوقتي بنبني 5-detector).

③ **اختيار الـ classifier:** `SGDClassifier` (خوارزمية classification).

`SGD` = Stochastic Gradient Descent. الفكرة الأساسية فيه إنه أثناء التدريب بيتعامل مع training instances **واحد واحد** بدل ما يعالج الداتا كلها مرة واحدة، وده بيخليه مناسب للتعامل مع datasets كبيرة ومناسب للـ online learning.

```
from sklearn.linear_model import SGDClassifier
sgd_clf = SGDClassifier(random_state=42)
sgd_clf.fit(X_train, y_train_5)
```

من كشكولك

④ **تدريب الـ classifier:** هنا بنقول للموديل: `X_train` (الصور) و `y_train_5` (هل كل صورة 5 أم لأ). فيتعلم العلاقة بين شكل الـ pixels وبين `True` (5) و `False` (مش 5). يعني بعد التدريب يقدر يشوف صورة جديدة ويقرر: هل دي 5؟

⑤ **استخدام الموديل للتنبؤ:** `sgd_clf.predict([some_digit])` ← لو طلع `True` = 5، ولو `False` = مش 5.

```
some_digit = X_train[y_train_5][0]                    # the first real '5' in the training set
print(sgd_clf.predict([some_digit]))
```

Output (real run)

```
[ True]        # decision score = 178.9 > 0
```

إضافة

في الكتاب الموديل توقع صح إن `some_digit` هو 5 (`array([True])`). وملحوظة: الـ `SGDClassifier` بيستخدم الـ `hinge loss` افتراضيًا فهو بيتصرف زي Linear SVM، ويقدر يبقى logistic regression لو عدّلت `loss="log_loss"`.

3 تقييم الـ Classifier: الـ Accuracy

من كشكولك

تقييم الـ classifier غالبًا بيكون أكتر تعقيدًا من تقييم الـ regressor، فالعديد من المقاييس لتقييم الأداء.

إحدى الطرق الجيدة لتقييم النموذج هي استخدام Cross-Validation. احتمال تلاقي إن الدقة (accuracy) عالية، لكن مش دايمًا ده معناه إن الموديل كويس. **مثال من الكتاب:** `dummy classifier` (مصنّف وهمي) بيصنّف كل حاجة في الفئة الأكتر تكرارًا (non-5): هيطلع accuracy حوالي **90%** لأن حوالي 10% بس من الصور هي 5، فهيكون صح في حوالي 90% من الوقت.

ده بيوضح إن الـ accuracy **مش** بتتعتبر مقياس الأداء المفضل للـ classifiers، خاصة عند التعامل مع مجموعات بيانات غير متوازنة (skewed)، أي لما تكون بعض الفئات أكتر شيوعًا من غيرها.

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

```
from sklearn.model_selection import cross_val_score
from sklearn.dummy import DummyClassifier

print(cross_val_score(sgd_clf, X_train, y_train_5, cv=3, scoring="accuracy"))
dummy_clf = DummyClassifier()                  # always predicts the most frequent class
print(cross_val_score(dummy_clf, X_train, y_train_5, cv=3, scoring="accuracy"))
```

```
Output (real run)
[0.9833 0.9896 0.9833]   # SGD
[0.8977 0.8998 0.8998]   # dummy: learns NOTHING and still gets ~90%
```

إضافة

من الكتاب (MNIST): SGD بيجيب `[0.95035, 0.96035, 0.9604]` (أكتر من 95% في كل fold!)، والـ dummy بيجيب `[0.90965, 0.90965, 0.90965]`. يعني الـ 95% بتاعة الـ SGD مش مبهرة قوي: الفرق عن موديل ما بيتعلمش حاجة مجرد 5%.

وفي التشغيل الحقيقي على digits: SGD ≈ 98.5% والـ dummy ≈ 89.9%. نفس الدرس: لازم تقارن بالـ dummy baseline.

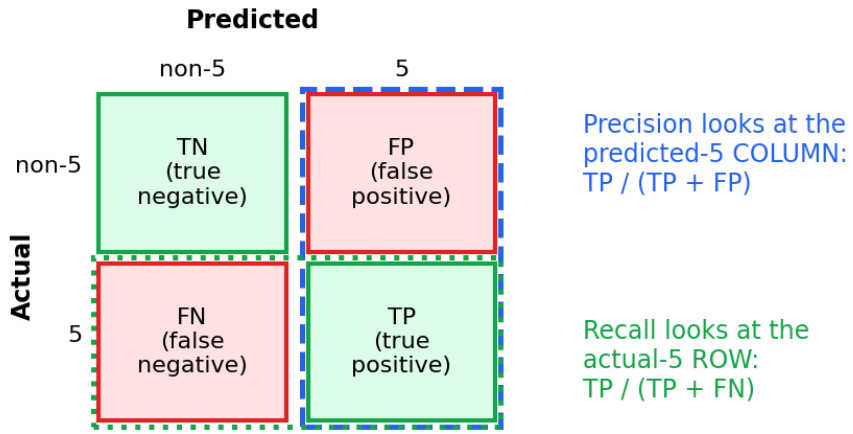
الكتاب كمان بيوريك طريقة تعمل بيها cross-validation بنفسك بـ `StratifiedKFold` (folds 3): كل fold بتتدرب على `clone` من الـ classifier وتقيس نسبة التنبؤات الصح، وبتدّي نفس الأرقام؛ مفيدة لو احتجت تحكّم أكتر في عملية الـ CV.

4 Confusion Matrix

من كشكولك

طريقة أفضل بكتير لتقييم أداء الـ classifier (Confusion Matrix). فكرتها: إننا بنعتبر إن عندنا Binary classifier بيجاوب 5 ولا مش 5، فكل صورة ليها حاجتين: **Actual** (هل هي فعلًا 5 ولا لأ) و**Predicted** (الموديل قال 5 ولا لأ). لما نقارن الاتنين نحصل على **4 حالات فقط**:

- `T` = True، `F` = False، `P` = Positive، `N` = Negative.
- الـ `CM` بتعدّ كام مرة الموديل غلط بين فئة A وفئة B لكل توليفة ممكنة. مثال من الكتاب: لو عايز تعرف كام مرة الموديل لخبط صورة "5" واعتبرها مش 5 (والعكس).



الصفوف = الحقيقة، والأعمدة = توقع الموديل. Precision بتبص على عمود الـ predicted-5، وRecall بتبص على صف الـ actual-5.

من كشكولك

عشان نحسب الـ `CM` لازم نعمل predictions الأول. نقدر نستخدم `cross_val_predict` :

```
from sklearn.model_selection import cross_val_predict
y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3)

from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_train_5, y_train_pred)
print(cm)
```

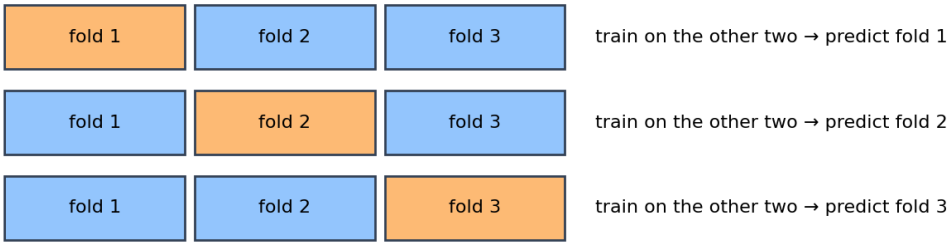
Output (real run)

```
[[1288    4]
 [  17  128]]
```

من كشكولك

انتبه: الـ `cross_val_predict` بتشتغل زي `cross_val_score` ، لكن بدل ما ترجع score بترجع **predictions** لكل instance في `X_train` . بتقسم البيانات لـ 3 folds: كل مرة بتتدرّب على جزأين وتعمل predictions على الجزء الثالث اللي الموديل ماشافهوش. وفي النهاية كل training instance بيبقى ليه out-of-sample prediction (من موديل ماتدرّبش على الـ instance دي)؛ وده مهم جدًا عشان الـ predictions تكون **عادلة** لما نستخدمها في تقييم الموديل.

بعد كده أصبح عندنا `y_train_5` وهو الـ actual labels، و `y_train_pred` وهو الـ predicted labels. الأرقام دي عشان تفهمها كل رقم بيدل على حاجة، وبتطلع معلومات كتيرة منها. لأن الـ `CM` بتوفر الكتير من المعلومات، ولكن في بعض الأحيان تفضل مقياسًا أكتر إيجازًا (Precision و Recall).



فكرة `cross_val_predict`: كل prediction جاي من موديل ماشافش الـ instance دي.

إضافة

قراءة الأرقام الحقيقية (digits): **TN = 1288** (non-5 وقال non-5 ✓)، **FP = 4** (non-5 وقال 5 ✗ "إنذار كاذب")، **FN = 17** (5 وقال non-5 ✗ "فوّت الـ 5")، **TP = 128** (5 وقال 5 ✓). المجموع = 1437 = كل الـ training set.

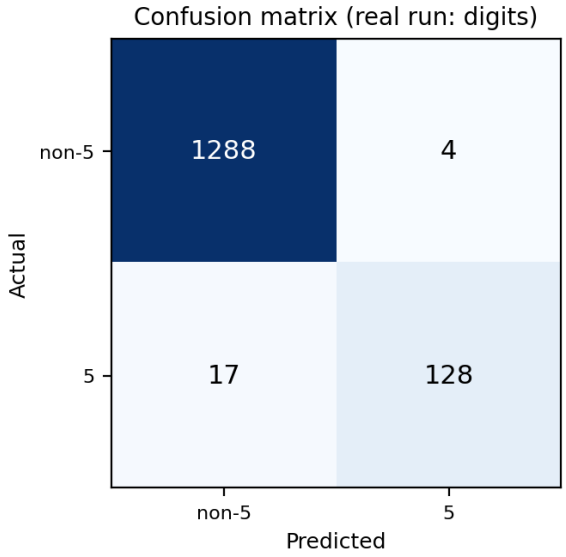
من الكتاب (MNIST، 60,000 صورة):

```
array([[53892,   687],
       [ 1891,  3530]])
```

- TN = 53,892 · FP = 687 · FN = 1,891 · TP = 3,530.

لو الموديل **مثالي** (`confusion_matrix(y_train_5, y_train_5)`) هيبقى عنده TP و TN بس، وكل الـ FP و FN أصفار:

```
Output (real run)
[[1292    0]
 [   0  145]]
```



الـ confusion matrix الحقيقية على digits.

5 Precision و Recall

من كشكولك

Precision (دقة الموديل في التنبؤات الإيجابية) هو دقة تنبؤات الـ classifier الإيجابية. يعني هو بيجاوب على سؤال: من كل الصور اللي الموديل قال إنها 5، كام واحدة كانت فعلًا 5؟ (الصح من التوقعات)

$$\text{precision} = \frac{TP}{TP + FP}$$

(كل اللي الموديل قال عليهم Positive = كل التوقعات.)

بس ده لوحده مش كفاية: المشكلة إنك ممكن تحقق `precision = 100%` بسهولة جدًا من غير ما الموديل يكون كويس. **مثال:** عندي 1000 صورة، منهم 100 فعلًا 5 و 900 مش 5، وعملت موديل غبي جدًا بيقولي كل الصور non-5 ماعدا صورة واحدة بس واثق جدًا إنها 5، والصورة فعلًا كانت 5 (`TP=1, FP=0` ← `precision = 1/(1+0) = 100%` . هل الموديل كويس؟ لأ طبعًا! هنا بييجي دور **Recall**.

Recall بيسأل سؤال مختلف: من كل الصور اللي هي فعلًا 5، الموديل قدر يكتشف كام واحدة؟ (يتسمى كمان true positive rate `TPR` أو sensitivity)

$$\text{recall} = \frac{TP}{TP + FN}$$

(كل الـ Positive الحقيقية.)

```
from sklearn.metrics import precision_score, recall_score
print(precision_score(y_train_5, y_train_pred))
print(recall_score(y_train_5, y_train_pred))
```

```
Output (real run)
0.9697
0.8828
```

من كشكولك

- الأول معناه: من كل الصور اللي الموديل قال عليها 5، حوالي **97.0%** كانوا فعلًا 5.
- التاني معناه: من كل الصور اللي هي فعلًا 5، الموديل قدر يكتشف حوالي **88.3%** منهم.

إضافة

الحساب من أرقام الـ CM: precision = 128 / (4 + 128) = **0.9697**، و recall = 128 / (17 + 128) = **0.8828** (نفس الـ output).

من الكتاب (MNIST): precision = 3530 / (687 + 3530) = **0.8371** و recall = 3530 / (1891 + 3530) = **0.6512**. لاحظ إن الموديل مش بيبان مبهر زي ما الـ accuracy (96%) كانت بتوحي: لما بيقول إنها 5 بيصيب 83.7% بس من المرات، وبيفوّت 35% من الـ 5. (على digits المشكلة أسهل فالأرقام أعلى.)

F1 Score 6

من كشكولك

أحيانًا محتاج رقم واحد يجمع الـ Precision و Recall: F1 score، وهو **المتوسط التوافقي** (Harmonic Mean) بين Precision و Recall.

$$F_1 = 2 \times \frac{\text{precision} \times \text{recall}}{\text{precision} + \text{recall}} = \frac{TP}{TP + \frac{FN + FP}{2}}$$

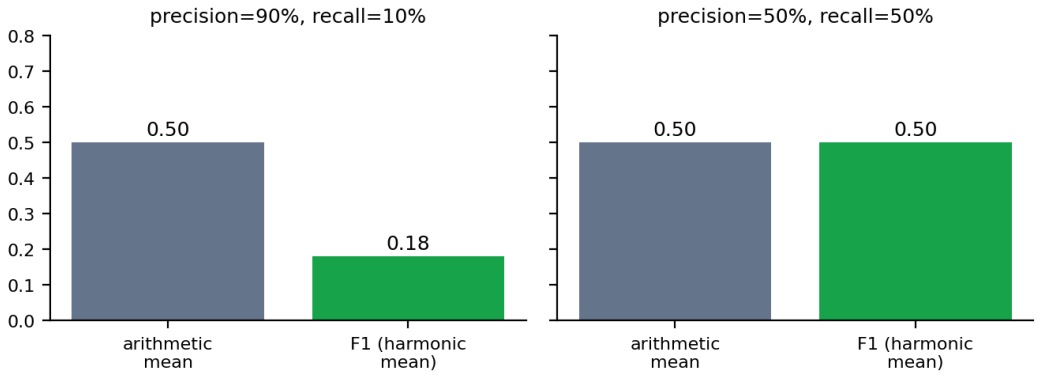
```
from sklearn.metrics import f1_score
f1_score(y_train_5, y_train_pred)
```

Output (real run)

```
0.9242
```

من كشكولك

ليه مش بنستخدم المتوسط العادي؟ لأن الـ F1 بيعاقب لو واحدة من Recall أو Precision منخفضة جدًا. مثلًا لو `Recall = 10%` و `Precision = 90%` المتوسط العادي هيقول 50% (وكأن الموديل كويس) لكن الـ F1 هيكون منخفض (حوالي 18%)، لأن الموديل لازم يكون كويس في الاتنين مع بعض عشان ياخد F1 عالي.



المتوسط العادي بيخدع (0.50)، لكن الـ F1 بيعاقب الاختلال بين precision و (0.18) recall.

من كشكولك

هل F1 دايمًا أفضل Metric؟ لأ. أحيانًا بتهتم بـ Precision وأحيانًا بـ Recall. **مثال:** عايز تتأكد إن الفيديو اللي الموديل بيسمح بيه آمن للأطفال؛ ممكن تقبل موديل بيرفض فيديوهات آمنة كتير (Recall واطي) بس ما يسمحش إلا بالفيديوهات الآمنة فعلًا (Precision عالي). هنا الـ Precision هو الأهم. وهكذا.

إضافة

- من الكتاب: F1 للـ SGD = **0.7325**. وهو بيفضّل الموديلات اللي precision و recall بتوعها متقاربين. (على digits: F1 = 0.924).
- مثال عكسي من الكتاب: لو بتبني نظام كشف سرقة من كاميرات المراقبة، الأفضل recall عالي (بتقبل إنذارات كاذبة كتير بس ماتفوّتش سرقة حقيقية) حتى لو الـ precision 30% بس.
- القاعدة: **مش ممكن تحصل على الاتنين مع بعض**، وده اللي اسمه Precision/Recall Trade-off (القسم الجاي).

7 Precision/Recall Trade-off

من كشكولك

زيادة الـ **Precision** غالبًا بتؤدي إلى انخفاض الـ **Recall**، والعكس. ليه؟ لأن الـ classifier عنده حاجة اسمها Threshold: الموديل مش بيقرر مباشرة 5 ولا مش 5، هو الأول بيحسب Score لكل صورة. مثلًا:

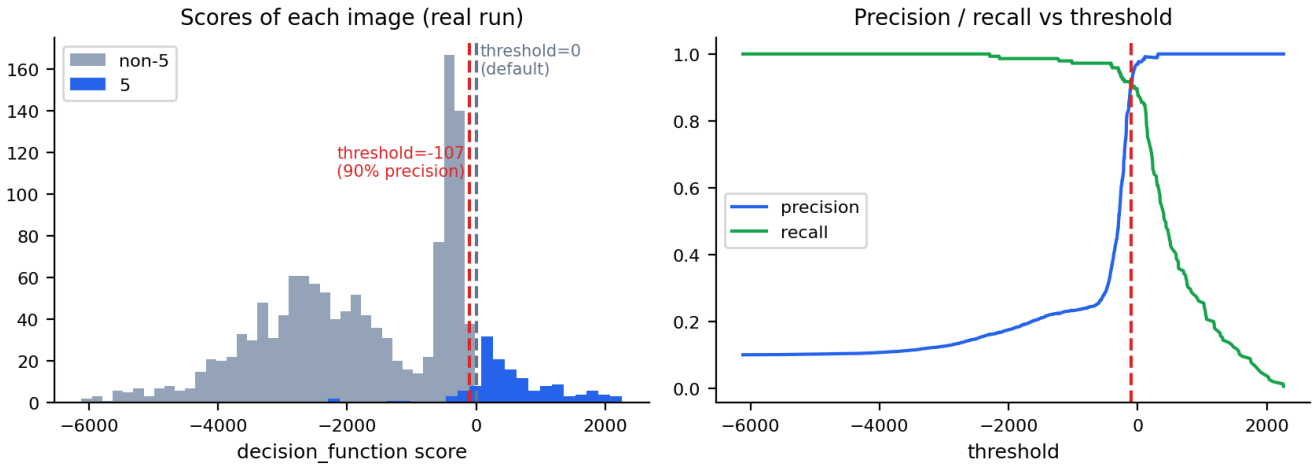
- الصورة A ← score = 5000
- الصورة B ← score = 3000
- الصورة C ← score = 1000
- الصورة D ← score = −500

وبعدين عندنا الـ Threshold (مثلًا `Threshold = 0`)، والقاعدة:

$$\hat{y} = \text{positive if score} > \text{threshold}, \quad \text{negative otherwise}$$

فلو رفعنا الـ Threshold لـ 3000، دلوقتي الموديل هيكون **أكتر تشددًا** ومش هيقول 5 إلا لو الـ score عالي جدًا.

- إذن لما Threshold **يزيد** ← الموديل هيقل عدد الـ False Positive ← **Precision يزيد** (غالبًا).
- لكن في المقابل ممكن يفوّت صور 5 حقيقية (False Negatives زادت) ← **Recall يقل**.
- والعكس صحيح: لو قلّلت الـ Threshold ← Recall يزيد و Precision غالبًا يقل.



من تشغيل حقيقي: شمال: توزيع الـ scores والخطوط المتقطعة = thresholds مختلفة. يمين: precision و recall بيتحركوا عكس بعض لما الـ threshold يتغير.

إضافة

نفس الفكرة بأرقام حقيقية (digits): لما بنرفع الـ threshold الـ precision بتطلع والـ recall بتنزل:

Threshold	Precision	Recall
1000-	23.3%	97.2%
300-	48.1%	95.2%
0	97.0%	88.3%
300	98.9%	61.4%
800	100.0%	29.7%
1500	100.0%	11.7%

من كشكولك

`decision_function()`: الكتاب بيستخدمها بدل `predict`. الفرق: `predict` بتدّيك القرار النهائي `True/False`، أما `decision_function` بتدّيك الـ **score** اللي الموديل بنى عليه القرار (مثلًا 2164)، وبعدها إنت اللي بتطبّق الـ threshold:

```
y_scores = sgd_clf.decision_function([some_digit])
threshold = 0
y_some_digit_pred = (y_scores > threshold)        # True
threshold = 3000                                    # (in the book; here use a large number)
y_some_digit_pred = (y_scores > threshold)        # False
```

من كشكولك

أنا بنفسي اخترت `threshold = 3000` وقلت: رغم إن الصورة كانت 5 فعلًا بقت `False` (الـ Recall قل). الـ `SGDClassifier` بيستخدم `threshold = 0` افتراضيًا، فـ `predict()` نفس نتيجة `(score > 0)`.

من كشكولك

اختيار الـ Threshold المناسب: مش بنختاره عشوائي. أول حاجة نجيب الـ scores لكل الـ training set بـ `cross_val_predict` مع `method="decision_function"`:

```
y_scores = cross_val_predict(sgd_clf, X_train, y_train_5, cv=3, method="decision_function")

from sklearn.metrics import precision_recall_curve
precisions, recalls, thresholds = precision_recall_curve(y_train_5, y_scores)
print(len(precisions), len(recalls), len(thresholds))
```

```
Output (real run)
1437 thresholds → precisions/recalls have one extra element (the last point, precision=1/recall=0)
```

من كشكولك

يعني بدل ما `cross_val_predict` ترجع `True/False`، طلبنا منها ترجع الـ scores. وبعد كده `precision_recall_curve` (function في sklearn) بتحسب Precision و Recall لكل Threshold ممكن؛ والـ arrays (`precisions` , `recalls` , `thresholds`) بتبقى بنفس الطول (تقريبًا: `precisions` و `recalls` أطول بعنصر واحد).

انتبه / تصحيح

الجدول اللي كتبته في كشكولك (Thresholds: −500, 0, 1000, 2000, 3000 مع Precision/Recall) واضح إنه **مثال توضيحي** لفكرة إن Precision بتزيد و Recall بتقل. أرقام الكتاب الحقيقية (MNIST) هي: للحصول على `precision = 90%` لازم `threshold ≈ 3370` والـ recall بيبقى حوالي **48%** (مش 50% بالظبط).

```
idx_for_90_precision = (precisions >= 0.90).argmax()      # first index where precision >= 90%
threshold_for_90_precision = thresholds[idx_for_90_precision]
y_train_pred_90 = (y_scores >= threshold_for_90_precision)
print(threshold_for_90_precision)
print(precision_score(y_train_5, y_train_pred_90), recall_score(y_train_5, y_train_pred_90))
```

```
Output (real run)
-106.98
0.9041 0.9103
```

إضافة

من الكتاب (MNIST): `threshold = 3370.0195`، `precision = 0.90003`، `recall = 0.47999`. يعني بالـ threshold ده: لما الموديل يقول "5" بيصيب 90% من المرات، لكنه بيكتشف 48% بس من الـ 5 الحقيقية. **والكتاب بيحذّر:** الـ classifier اللي precision بتاعه عالي بس recall بتاعه واطي **مش مفيد**. ولو حد قالك "خلينا نوصل 99% precision" لازم تسأل: عند أنهي recall؟

على digits: `threshold = -107.0` بيدّي 90.4% precision و 91.0% recall (أحسن بكتير من MNIST لأن digits مشكلة أسهل). `argmax()` هنا بترجع أول موضع فيه `True` (أول threshold بيحقق precision ≥ 90%).

8 ROC Curve و ROC AUC

من كشكولك

ROC Curve: هي بتربط الـ `TPR` (Recall) بالـ `FPR` (False Positive Rate). الـ FPR بيجاوب: من كل الـ Negative الحقيقية، كام واحد الموديل غلط وقال عليهم Positive؟

$$\text{FPR} = \frac{FP}{FP + TN} = 1 - \text{TNR}$$

`TNR` = **True Negative Rate** يعني: من كل الـ Negative، كام واحد الموديل عرفه صح وقال عليه Negative، وبتتسمى كمان Specificity. والـ FPR مرتبط بيها: `FPR = 1 − TNR`.

$$\text{TNR (specificity)} = \frac{TN}{TN + FP}$$

- ROC: `X-axis` ← **FPR**، و `Y-axis` ← **TPR (Recall)**.

```
from sklearn.metrics import roc_curve
fpr, tpr, thresholds = roc_curve(y_train_5, y_scores)
```

من كشكولك

⇒ بتحسب `TPR + FPR` لـ Thresholds مختلفة. ولو الـ Threshold قلّ ⇒ الموديل هيعتبر حالات أكتر Positive، فـ `TPR` (Recall) و `FPR` الاتنين يزيدوا (والعكس).

انتبه / تصحيح

في كشكولك كتبت "Y-axis" للـ ROC مرتين؛ التصحيح: الـ **X-axis = FPR** والـ **Y-axis = TPR**.

من كشكولك

ROC AUC: بدل ما تقارن بين الـ curves بعينك، ممكن تقيس أداء الـ classifier برقم واحد: الـ **Area Under the Curve** (المساحة تحت الـ ROC curve) قيمتها:

- `1.0` ← perfect classifier (مثالي).
- `0.5` ← random classifier (عشوائي، الخط المتقطع القطري).

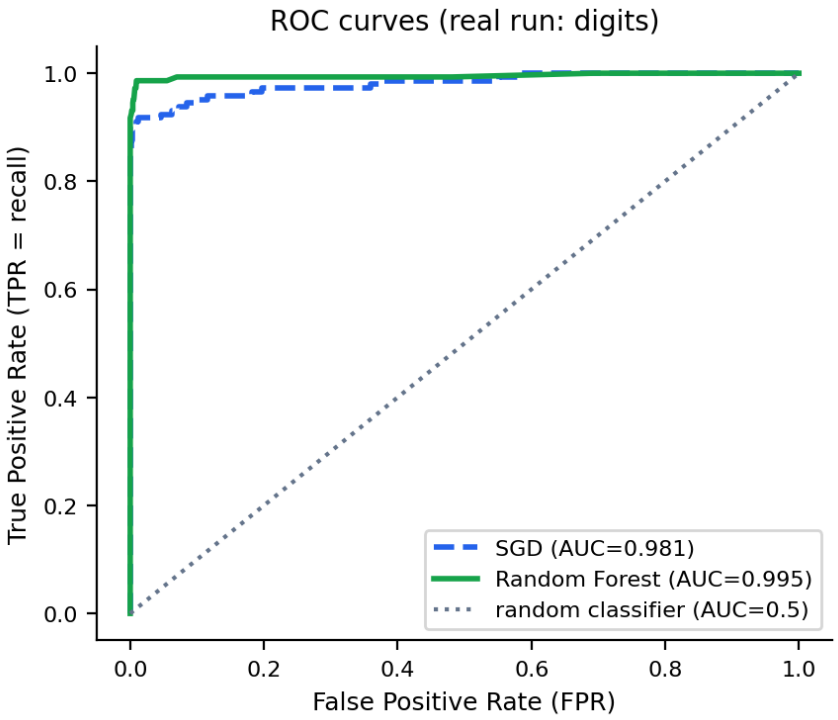
```
from sklearn.metrics import roc_auc_score
roc_auc_score(y_train_5, y_scores)
```

Output (real run)

```
0.9807
```

من كشكولك

إمتى أستخدم ROC AUC وإمتى PR curve؟ استخدم **PR curve** لما الـ class positive **نادر** أو لما تهتم بالـ False Positives أكتر من الـ False Negatives. غير كده ممكن تكون الـ ROC مناسبة.



منحنى ROC (تشغيل حقيقي): كل ما قرب من الركن الشمال-فوق كل ما أحسن.

إضافة

سبب إن الـ ROC ممكن تضلّل لما الـ positives نادرة (من الكتاب): في مثالنا الـ ROC AUC للـ SGD = **0.96** "عالي" لكن ده بيبان كده لأن الـ 5 قليلة (10% بس) مقارنة بالـ non-5، فالـ FPR بتبقى صغيرة حتى لو الموديل بيغلط كتير في الـ positive. أما الـ PR curve فبتوريك إن في مجال للتحسين. وكل ما الـ curve أقرب للركن الأيمن العلوي كل ما أحسن.

9 مقارنة مع Random Forest

من كشكولك

دلوقتي هنجرب classifier مختلف: **Random Forest**. في الأول كنا بنشتغل بـ SGD classifier. الهدف هنا مش إننا نبدأ موديل جديد وخلاص؛ الهدف هو **مقارنة أداء Random Forest بالـ SGD**.

```
from sklearn.ensemble import RandomForestClassifier
forest_clf = RandomForestClassifier(random_state=42)
y_probas_forest = cross_val_predict(forest_clf, X_train, y_train_5, cv=3, method="predict_proba")
print(y_probas_forest[:3])
```

Output (real run)

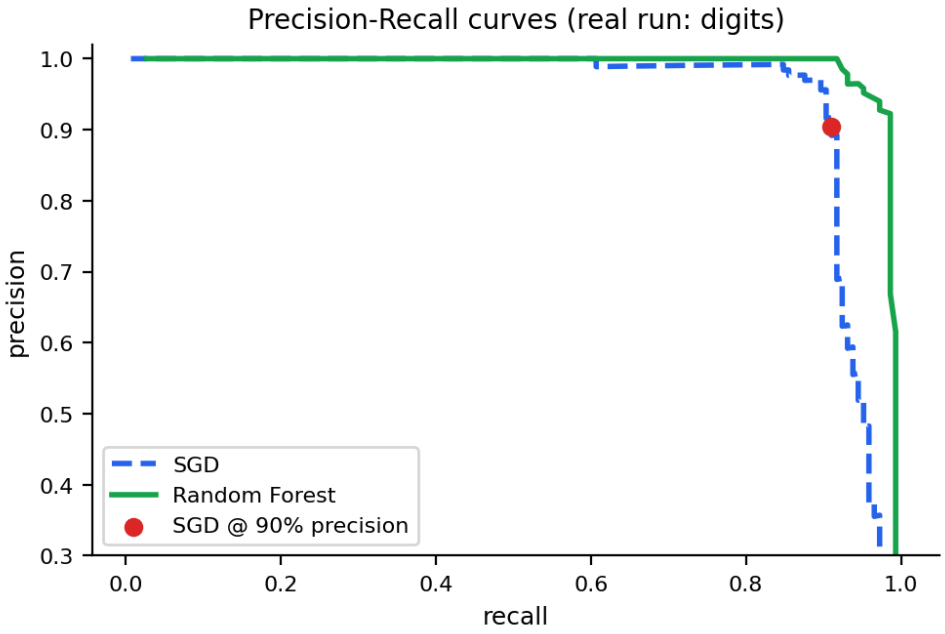
```
[[0.76 0.24]
 [0.96 0.04]
 [0.03 0.97]]
```

من كشكولك

الـ Random Forest ملهوش `decision_function()`؛ بدلها عنده `predict_proba()` وبترجع **احتمالات** (تقدير للثقة). بدل ما ترجع `True/False` بترجع احتمال كل فئة، مثلًا `[0.11, 0.89]` يعني: probability of negative = 11% و positive = 89%. وبالتالي الموديل شايف إن احتمال إنها 5 = 89%.

`y_probas_forest` شكلها تقريبًا: `[[0.11, 0.89], [0.99, 0.01], ...]` ← **Column 0** = probability of negative، **Column 1** = probability of positive. واحنا مهتمين بالـ Positive = 5، فبناخد العمود رقم 1 (`[:, 1]`) وده بقى الـ scores اللي هنستخدمها. وبعدها نفس قصة PR:

```
y_scores_forest = y_probas_forest[:, 1]          # column 1 from every row = P(positive)
precisions_forest, recalls_forest, thresholds_forest = precision_recall_curve(y_train_5, y_scores_forest)
```



منحنى Precision-Recall (تشغيل حقيقي): الـ Random Forest (أخضر) أقرب بكتير للركن العلوي الأيمن من الـ SGD (أزرق متقطع).

إضافة

نتايج حقيقية على digits: Random Forest عند threshold 50%: F1 = **0.897**، precision = **1.000**، recall = **0.814**، ROC AUC = **0.9952** (مقابل 0.9807 للـ SGD).

نتايج الكتاب (MNIST): الـ PR curve بتاعة الـ Random Forest أقرب بكتير للركن العلوي الأيمن من الـ SGD. وعند threshold: F1 = 0.924 50% و ROC AUC = 0.998 (مقابل 0.7325 و 0.960 للـ SGD)، وتقريبًا precision 99.1% و recall 86.6%.

تحذير من الكتاب: دي احتمالات مُقدَّرة (estimated) مش حقيقية. مثلًا الصور اللي الموديل قدّر احتمالها بين 50% و 60% positive، كان حوالي 94% منها فعلًا positive، يعني الموديل كان متحفظ زيادة. وفي `sklearn.calibration` أدوات تعمل calibrate للاحتمالات.

Multiclass Classification 10

من كشكولك

احنا كنا في الـ Binary classification: 5 أو non-5. دلوقتي **Multiclass classification** يعني عندنا 10 classes (0 → 9)؛ يعني classifier قادر يميّز بين أكتر من class.

هل كل classifier يقدر يعمل Multiclass؟ في scikit-learn فيه classifiers تقدر تتعامل مع Multiclass مباشرة (natively) زي: `LogisticRegression` و `RandomForestClassifier` و `GaussianNB`. وفي classifiers تانية هي Binary فقط، زي: `SGDClassifier` و `SVC`.

لكن تقدر تستخدم classifiers Binary لعمل Multiclass باستخدام استراتيجيات معينة، وده نقطة مهمة: scikit-learn بتحوّل الـ classifier Binary تلقائيًا لـ problem Multiclass.

من كشكولك

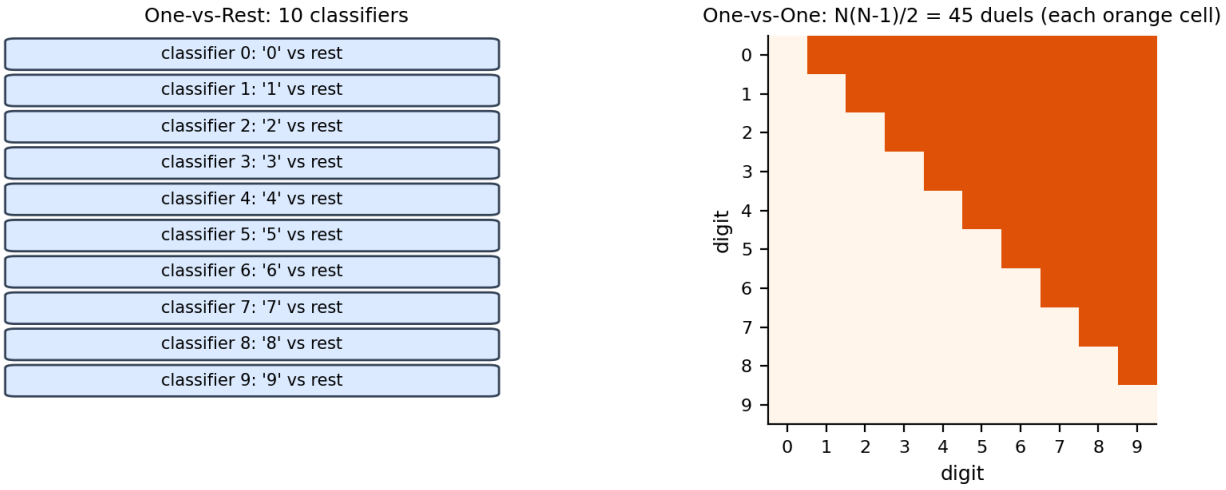
① **One-vs-Rest (OvR)** (أو One-vs-All OvA): دي أول استراتيجية. عندنا 10 أرقام (0 → 9). بدل classifier واحد يعرف العشرة، نعمل **Binary classifiers 10**، كل واحد يعرف رقم واحد: Classifier 0: 0 vs الباقي، Classifier 1: 1 vs الباقي، ... وكده. لما تيجي صورة جديدة بنشغّل الـ 10 classifiers كلهم، كل واحد يطلع score، ونختار الـ classifier اللي طلّع أعلى score.

② **One-vs-One (OvO):** الاستراتيجية التانية مختلفة. بدل رقم واحد ضد كل الباقي، نعمل classifier لكل **زوج** من الأرقام: 0 vs 1، 0 vs 2، ... 1 vs 2، 1 vs 3، ... 8 vs 9. ولو عندنا N classes عدد الـ classifiers =

$$\text{number of classifiers} = \frac{N(N-1)}{2}$$

في MNIST = **Binary classifiers 45**. ولما تيجي صورة جديدة، ندخلها على كل الـ 45 classifier، وكل classifier بيعمل "مواجهة" بين رقمين، والرقم اللي يكسب أكبر عدد من المواجهات هو الـ prediction النهائي.

طيب ليه أستخدم OvO بدل OvR؟ كل classifier في OvO بيتدرب على **جزء صغير** من الداتا (بس صور الرقمين) فمش محتاج الـ training set كله. ده مفيد مع بعض الخوارزميات (زي **SVM**) اللي بتبطأ كتير لما حجم الـ training set يكبر. لذلك الكتاب بيقول: مع بعض الخوارزميات، تدريب عدد كبير من classifiers على datasets صغيرة أسرع من تدريب عدد قليل على dataset ضخمة. ومع معظم الـ Binary classifiers، يكون **OvR هو الاختيار الأفضل**.



OvR: 10 classifiers (كل واحد رقم ضد الباقي). OvO: 45 مواجهة (كل خلية برتقالي = زوج).

من كشكولك

لو استخدمت Binary classifier في scikit-learn، Multiclass problem بتكتشف ده وتستخدم OvO أو OvR حسب الخوارزمية (الكتاب بيقول كده).

مثال SVC:

```
from sklearn.svm import SVC
svm_clf = SVC(random_state=42)
svm_clf.fit(X_train[:2000], y_train[:2000])       # y_train NOT y_train_5: the original 10-class target
print(svm_clf.predict([some_digit]))
```

Output (real run)
```
[5]       # (on digits we trained on the first 600 images)
```

من كشكولك

- لاحظ إنه قال `y_train` وليس `y_train_5`، لأننا بنعمل Multiclass للمشكلة الأصلية (0 → 9).
- وبما إن الـ `SVC` هو Binary classifier (5 مش 5)، scikit-learn استخدمت **OvO** وقدّرت تدرّب **Binary 45 classifiers**. والنتيجة: `array(['5'])` يعني الصورة اتصنفت على إنها 5، لكن داخليًا مش classifier واحد اللي قال 5؛ كان في 45 مواجهة والـ 5 كسبت أكبر عدد منها.

من كشكولك

`decision_function()` **في Multiclass:** مع الـ Binary classifier كنت بتاخد score واحد: هل 5؟ لكن هنا عندنا 10 classes فترجع **10 scores** (score لكل class):

```
some_digit_scores = svm_clf.decision_function([some_digit])
print(some_digit_scores.round(2))
class_id = some_digit_scores.argmax()        # index of the biggest score
print(class_id, svm_clf.classes_[class_id])  # classes_[class_id] = the actual class label
```

Output (real run)
```
[[ 6.14  1.89  2.85  7.23  0.72  9.28 -0.29  3.99  5.18  8.27]]
5 5
```

من كشكولك

الكتاب عرض: `[3.79, 0.73, ..., 9.30, ...]` ← 10 scores، وأعلى score كان `9.30` وهو الـ index رقم 5 إذن الـ Prediction = 5. `argmax()` بترجع الـ **index** بتاع أكبر قيمة. وبعدين `classes_[class_id]` عشان تجيب الـ label الفعلي (في MNIST الـ index بيطابق الرقم صدفة، لكن في العموم لازم تعمل lookup).

إضافة

ولو عايز تشوف الـ 45 مواجهة نفسها اطلب `SVC(decision_function_shape="ovo")` : الشكل بقى `(1, 45)` (45 score) بدل `(1, 10)` (10 scores). **الـ score في OvO (من الكتاب):** كل class بياخد score = عدد المواجهات اللي كسبها زائد/ناقص تعديل صغير (≥ 0.33) لكسر التعادل.

من كشكولك

طب لو عايز أفرض OvO أو OvR بنفسي (scikit-learn ما تقررش)؟

```
from sklearn.multiclass import OneVsRestClassifier       # or OneVsOneClassifier
ovr_clf = OneVsRestClassifier(SVC(random_state=42))
ovr_clf.fit(X_train[:2000], y_train[:2000])
print(len(ovr_clf.estimators_))
```

```
Output (real run)
10
```

من كشكولك

هنا قلت صراحة: استخدم `SVC` لكن حوّلها إلى Multiclass باستخدام OvR (فبيتدرّب 10 classifiers).

إضافة

الـ `SGDClassifier` بيستخدم OvR **تلقائيًا** مع Multiclass: `sgd_clf.fit(X_train, y_train)` بيدّي 10 scores.

من الكتاب (MNIST): الموديل ده بيتوقع `'3'` للـ `some_digit` غلط (رغم إنه 5)، وبالـ `cross_val_score` بيطلع accuracy حوالي **87%** (0.8689, 0.85835, 0.87365)؛ ولو عملت `StandardScaler` الأول بيرتفع لحوالي **90%** (0.8983, 0.891, 0.9018). يعني الـ scaling بتفرق هنا برضه.

على digits: accuracy ≈ 92.2% بدون scaling و ≈ 94.3% بعد `StandardScaler`. وللمقارنة: التخمين العشوائي في 10 classes = 10% بس.

11 Error Analysis

من كشكولك

Error Analysis: هو لما نسأل الموديل يغلط في إيه بالظبط؟ يعني تحليل أنواع الأخطاء بدل ما تبص على رقم الـ accuracy فقط.

في الـ Multiclass، الـ `CM` مش بقت `TP/FN/FP/TN` زي الـ Binary، لأن بقى عندنا 10 classes، فالـ `CM` هتبقى **10 × 10**: `Row = Actual class`، `Columns = Predicted class`.

`ConfusionMatrixDisplay`: الكتاب بيعمل كده:

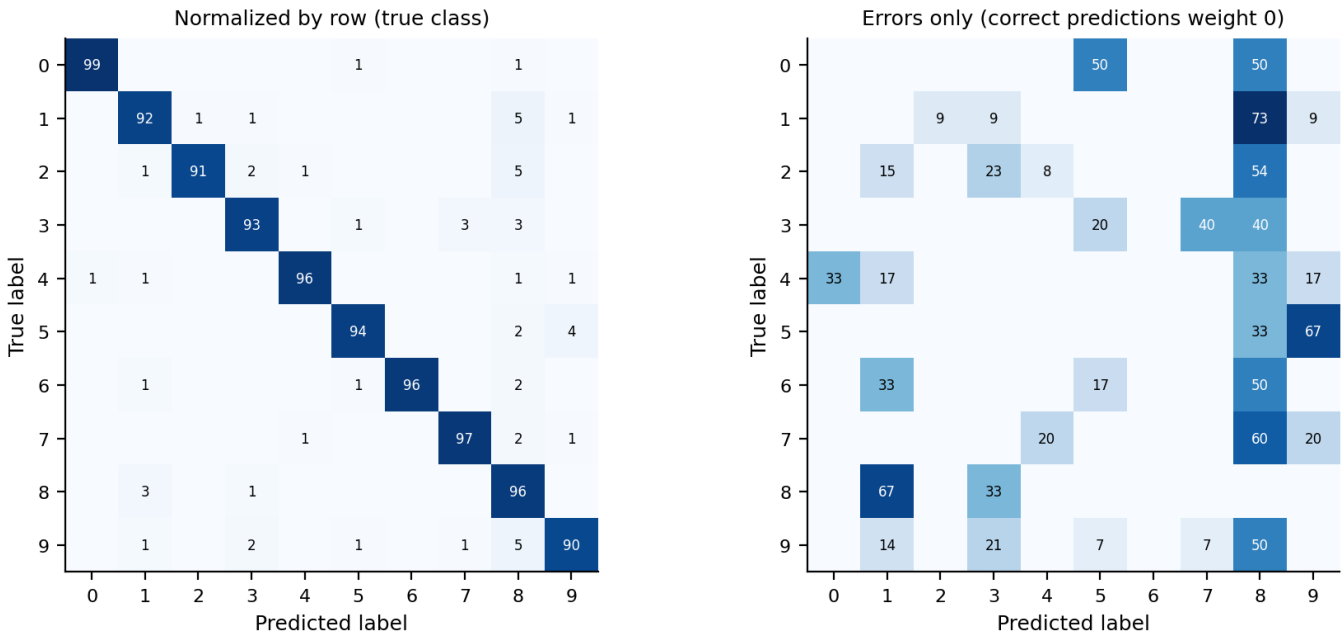
```
y_train_pred = cross_val_predict(sgd_clf, X_train_scaled, y_train, cv=3)

from sklearn.metrics import ConfusionMatrixDisplay
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred)

# better: add normalize and values_format
ConfusionMatrixDisplay.from_predictions(y_train, y_train_pred, normalize="true", values_format=".0%")
```

من كشكولك

- وده بيرسم الـ `CM` بشكل صورة بدل ما تتعامل مع أرقام كتيرة.
- **إزاي تقرأ الـ CM؟** الأرقام الموجودة على الـ **Main Diagonal** = التوقعات الصحيحة. وكل ما كان الـ Main Diagonal أغمق، كان الموديل أفضل عمومًا.
- `normalize="true"` + `values_format=".0%"`: بتحوّل الأرقام لنسب (مقسومة على مجموع الصف) فتكون أوضح، لأن بعض الـ classes أكتر عددًا من غيرها.



تشغيل حقيقي على digits (accuracy الكلية 94%): شمال النسب لكل class، يمين الأخطاء بس (القطر = صفر). أكتر خطأ: الرقم 1 بيتلخبط مع 8 (5%).

إضافة

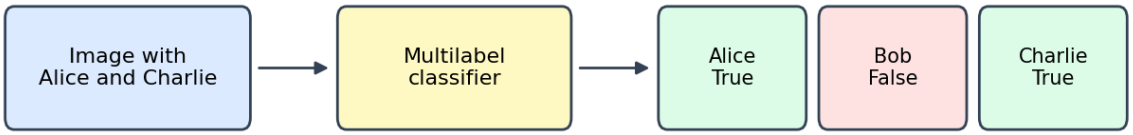
اللي اكتشفه الكتاب من الـ CM بعد الـ normalize على MNIST:

- فقط **82%** من صور الـ 5 اتصنفت صح. وأكتر خطأ مع الـ 5 كان تصنيفها 8 (حصل مع **10%** من الـ 5). لكن 2% بس من الـ 8 اتصنفت 5؛ يعني الـ confusion matrix **مش متماثلة** (symmetrical).
- لو عايز الأخطاء تبان أكتر: `sample_weight=(y_train_pred != y_train)` بيحط وزن صفر على التوقعات الصحيحة. هنا عمود الـ **8** بيبقى مضيء جدًا، يعني كتير من الأرقام اتصنفت 8 غلط (أكتر خطأ شائع لأغلب الـ classes).
- **انتبه:** النسب في نسخة "الأخطاء بس" مش نسبة من كل الصور؛ مثلًا 36% في صف 7 عمود 9 معناها 36% من **أخطاء** الـ 7 كانت تصنيفها 9، مش 36% من كل الـ 7. (الحقيقة 3% بس من الـ 7 اتصنفت 9.) و `normalize="pred"` بيطبّع على العمود.
- الاستنتاج في الكتاب: ركّز على تقليل الـ **8 الكاذبة** مثلًا بجمع داتا أكتر لأرقام شكلها شبه الـ 8 (بس مش 8) عشان الموديل يتعلم يفرق، أو تعمل features جديدة (زي عدد الـ closed loops)، أو تعمل preprocessing للصور.

12 Multilabel Classification

من كشكولك

طب لو معايا صورة وعايز أتعرف على أكتر من حاجة؟ يعني مثلًا: معايا classifier بيتعرف على 3 أشخاص (Alice, Bob, Charlie)، وصورة فيها Alice و Charlie، النتيجة `[True, False, True]` . إذن الـ instance الواحد ممكن يطلع له أكتر من label Binary في نفس الوقت = **Multilabel classification**.



صورة واحدة → أكتر من label (كل label قيمته True/False).

من كشكولك

المثال بتاع MNIST في الكتاب: عمل label لكل صورة: الـ label الأول: هل الرقم **كبير**؟ والـ label التاني: هل الرقم **فردي**؟

```
import numpy as np
y_train_large = (y_train >= '7')                    # 7, 8, 9 -> True | 0..6 -> False
y_train_odd = (y_train.astype('int8') % 2 == 1)     # 1,3,5,7,9 -> True | 0,2,4,6,8 -> False
y_multilabel = np.c_[y_train_large, y_train_odd]    # stack the 2 labels as 2 columns
```

من كشكولك

مثلًا لو الرقم 5: `large = False` و `odd = True` ← `[False, True]`. فأصبح عندي لكل صورة **targets 2** بدل target واحد.

```
from sklearn.neighbors import KNeighborsClassifier
knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train, y_multilabel)
print(knn_clf.predict([some_digit]))
```

Output (real run)

```
[[False True]]        # 5 is NOT large, and it IS odd  ✓
```

من كشكولك

هنا الـ KNN بيتعلم من `X_train` : `[Large?, Odd?]` وليس رقم واحد.

تقييم الـ Multilabel classifier: مفيش Metric واحدة لازم تستخدمها دايمًا؛ اختيار الـ Metric بيعتمد على المشروع. الكتاب اقترح: احسب F1 لكل label وبعدها خد المتوسط:

```
y_train_knn_pred = cross_val_predict(knn_clf, X_train, y_multilabel, cv=3)
f1_score(y_multilabel, y_train_knn_pred, average="macro")
```

Output (real run)

```
0.9780      # (book on MNIST: 0.9764)
```

من كشكولك

- `average="macro"` : احسب F1 لكل label ثم خد المتوسط، وبيدّي كل label **نفس الوزن**.
- المشكلة في macro: لو عدد الحالات مختلف جدًا (مثلًا صور Alice كتير، و Bob قليلة، و Charlie صور قليلة جدًا) ممكن تحب تدّي label اللي ليه حالات أكتر وزن أكبر. فتستخدم `average="weighted"` : كل label ياخد وزنه حسب **support** (أي عدد الـ instances اللي تنتمي إليها).

إضافة

على digits: F1 لكل label = 0.967 (large) و 0.989 (odd). الـ macro = **0.9780** والـ weighted = **0.9808**؛ قريبين لأن الـ labels متوازنين تقريبًا. الفرق بيكبر لما يبقى فيه label نادر.

من كشكولك

ماذا لو الـ classifier ما بيدعمش Multilabel مباشرة؟ (SVC مثلًا). ممكن تدرّب موديل منفصل لكل label، لكن ساعتها تظهر مشكلة إن الـ models مش بتعرف حاجة عن بعض (ما بتشوفش ما الذي توقعته الموديلات الأخرى). لحل المشكلة، الكتاب بيقترح **تنظيم الـ models في سلسلة (Chain): Classifier Chain**.

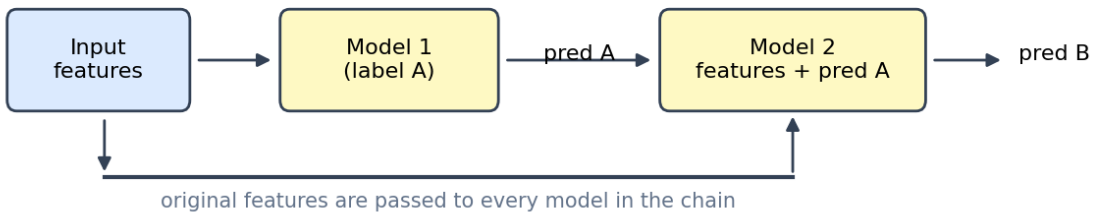
الفكرة: `Input` ← `Model 1` ← `Prediction 1`؛ والموديل التاني بيعتمد على: **original features +** **predictions from previous models**. وبالتالي الموديلات الموجودة في السلسلة تستطيع الاستفادة من العلاقات بين الـ labels.

```
from sklearn.multioutput import ClassifierChain
chain_clf = ClassifierChain(SVC(), cv=3, random_state=42)
chain_clf.fit(X_train[:2000], y_multilabel[:2000])
print(chain_clf.predict([some_digit]))
```

```
Output (real run)
[[False True]]
```

من كشكولك

هنا أنشأنا Chain Classifier باستخدام SVC، والـ `cv=3` بيخلي السلسلة تستخدم cross-validation للحصول على clean out-of-sample predictions أثناء التدريب، بدل ما تعتمد مباشرة على نتايج جاية من نفس البيانات اللي اتدرّبت عليها النماذج السابقة.



Classifier Chain: كل موديل بياخد الـ features الأصلية + توقعات الموديلات اللي قبله.

Multioutput Classification 13

من كشكولك

Multioutput-multiclass classification وهي تعميم لـ Multilabel classification. الفرق الأساسي: في الـ Multilabel كل label عنده **قيمتين فقط** (True/False)، لكن في الـ Multioutput كل output ممكن يكون عنده **أكتر من قيمتين** (يعني كل label نفسه Multiclass).

مثال الكتاب: Image Denoising (تنظيف الصور): الـ input = صورة مشوشة (noisy)، والـ output = صورة نظيفة. وتدرّب الموديل على: Noisy Image → Clean Image. الـ output فيه 784 pixels، وكل pixel هو output منفصل، وكل output ممكن ياخد قيمة من 0 لـ 255 (يعني قيم كتيرة). إذن عندنا Multiple outputs ولكل output قيم كتيرة: **Multioutput classification**.

```
noise = np.random.randint(0, 100, (len(X_train), 784))
X_train_mod = X_train + noise                    # noisy images = the INPUT
y_train_mod = X_train                            # clean images = the OUTPUT (784 outputs!)

knn_clf = KNeighborsClassifier()
knn_clf.fit(X_train_mod, y_train_mod)
clean_digit = knn_clf.predict([X_test_mod[0]])
```



تشغيل حقيقي على digits (KNN، الأخطاء بالـ MSE): بعد التنظيف الخطأ نزل من 9.1 لـ 3.4. (صف 1: المشوّشة، صف 2: ناتج الموديل، صف 3: النضيفة المطلوبة).

إضافة

النوع	عدد الـ outputs	قيم كل output	مثال
Binary	1	قيمتين	5 أو لأ
Multiclass	1	أكتر من قيمتين	الرقم من 0 لـ 9
Multilabel	أكتر من 1	قيمتين لكل واحد	Alice, Bob, Charlie (True/False)
Multioutput	أكتر من 1	أكتر من قيمتين لكل واحد	كل pixel (0-255) في الصورة النضيفة

الخط الفاصل بين Classification و Regression ضبابي هنا: التنبؤ بقيمة pixel أقرب للـ regression لكن الكتاب بيعتبره classification لأنه multioutput والقيم محدودة.

14 أسئلة مراجعة سريعة

جاوب الأول في دماغك، وبعدين شوف الإجابات في الصفحة اللي بعدها.

1. إيه الفرق بين Classification و Regression؟
2. ليه حوّلنا الـ labels لـ (True/False) y_train_5؟
3. إيه هو الـ SGDClassifier وليه مناسب للداتا الكبيرة؟
4. ليه الـ accuracy مش مقياس كويس للـ classifiers؟ مثال (dummy).
5. إيه الفرق بين cross_val_score و cross_val_predict؟
6. اشرح الـ Confusion Matrix (TN, FP, FN, TP).
7. اكتب معادلة Precision و Recall وفسّرهم.
8. مثال إزاي ممكن تحقق precision 100% بموديل وحش؟
9. ليه F1 بيستخدم المتوسط التوافقي؟
10. إمتى أفضّل Precision، وإمتى Recall؟
11. اشرح الـ Precision/Recall trade-off والـ threshold.
12. الفرق بين predict و decision_function؟
13. إزاي تختار threshold لـ precision 90%؟
14. إيه هو الـ ROC curve؟ وإيه معنى FPR و TNR؟
15. إيه معنى ROC AUC = 1 و 0.5؟
16. إمتى أستخدم PR curve بدل ROC؟
17. إيه الفرق بين decision_function و predict_proba؟ وإزاي بنقارن Random Forest بالـ SGD؟
18. اذكر 3 classifiers بتدعم multiclass مباشرة وإتنين binary فقط.
19. اشرح OvR و OvO وكام classifier في MNIST؟
20. إزاي بتطلع الـ class من decision_function في multiclass؟
21. إيه هو Error Analysis؟ وإزاي تقرأ الـ CM؟
22. إيه الفرق بين Multilabel و Multioutput؟
23. إزاي تقيّم Multilabel؟ ومتى macro ومتى weighted؟
24. إيه هو Classifier Chain؟

15 إجابات أسئلة المراجعة

1. Classification بيتوقع فئة، و Regression بيتوقع رقم.
2. عشان نبني binary classifier (5 ولا مش 5)، فالموديل بيحتاج يعرف هل الصورة 5 بس.
3. بيتعلم من الـ instances واحد واحد (Stochastic Gradient Descent)، فمناسب للـ datasets الكبيرة والـ online learning.
4. في الداتا غير المتوازنة (skewed): dummy classifier بيقول 'مش 5' دايمًا بيطلع accuracy حوالي 90% (0.90965 في الكتاب) من غير ما يتعلم حاجة.
5. الأولى بترجع scores للـ folds، والتانية بترجع out-of-sample predictions لكل instance.
6. الصفوف = الحقيقة والأعمدة = التوقع. TN و TP صح، FP إنذار كاذب، FN فوّت حالة positive.
7. precision = TP/(TP+FP): من اللي قال عليه 5 كام فعلًا 5. recall = TP/(TP+FN): من الـ 5 الحقيقية كام اكتشف.
8. موديل بيتوقع positive لصورة واحدة بس واثق منها وصح: TP=1, FP=0، لكن بيفوّت باقي الـ 5 (recall واطي جدًا).
9. عشان يعاقب لو precision أو recall واطي جدًا؛ لازم الاتنين يبقوا عاليين.
10. Precision لو الـ false positive غالي (فيديوهات آمنة للأطفال). Recall لو الـ false negative غالي (كشف سرقة/مرض).
11. الموديل بيحسب score وبنقارنه بـ threshold. رفعه = precision غالبًا يزيد و recall يقل، وخفضه العكس.
12. predict بترجع القرار النهائي (True/False)، و decision_function بترجع الـ score وإنت بتحدد الـ threshold.
13. cross_val_predict(method='decision_function') ثم precision_recall_curve، وبعدين (precisions >= 0.90).argmax() (الكتاب: recall ≈ 48% → threshold ≈ 3370).
14. منحنى TPR (على Y) ضد FPR (على X). FPR = FP/(FP+TN) = 1 − TNR، وTNR = specificity.
15. classifier = 1 مثالي، 0.5 = عشوائي.
16. لما الـ class positive نادر أو لما الـ false positives أهم من الـ false negatives.
17. Random Forest مفيهوش decision_function، فبنستخدم predict_proba[:, 1] كـ scores ونقارن PR/ROC.
18. LogisticRegression, RandomForestClassifier, GaussianNB (natively). SGDClassifier, SVC (binary فقط).
19. OvR: 10 classifiers (رقم ضد الباقي). OvO: N(N−1)/2 = 45 (زوج ضد زوج). OvO أحسن مع SVM لأنه بيتدرب على أجزاء صغيرة.
20. argmax() بيدّي الـ index، و classes_[class_id] بيدّي الـ label.
21. تحليل أنواع الأخطاء. الـ Main Diagonal = التوقعات الصحيحة؛ بنعمل normalize='true' وبنشيل الـ diagonal (sample_weight) عشان نشوف الأخطاء.
22. Multilabel: كل label قيمتين (True/False). Multioutput: كل output ممكن يكون multiclass (زي كل pixel 0-255).
23. F1 لكل label ثم متوسط. macro = نفس الوزن، weighted = حسب عدد الحالات (support) لو الـ labels غير متوازنة.
24. سلسلة موديلات: كل موديل بياخد الـ features الأصلية + توقعات اللي قبله، فيستفيد من العلاقة بين الـ labels.